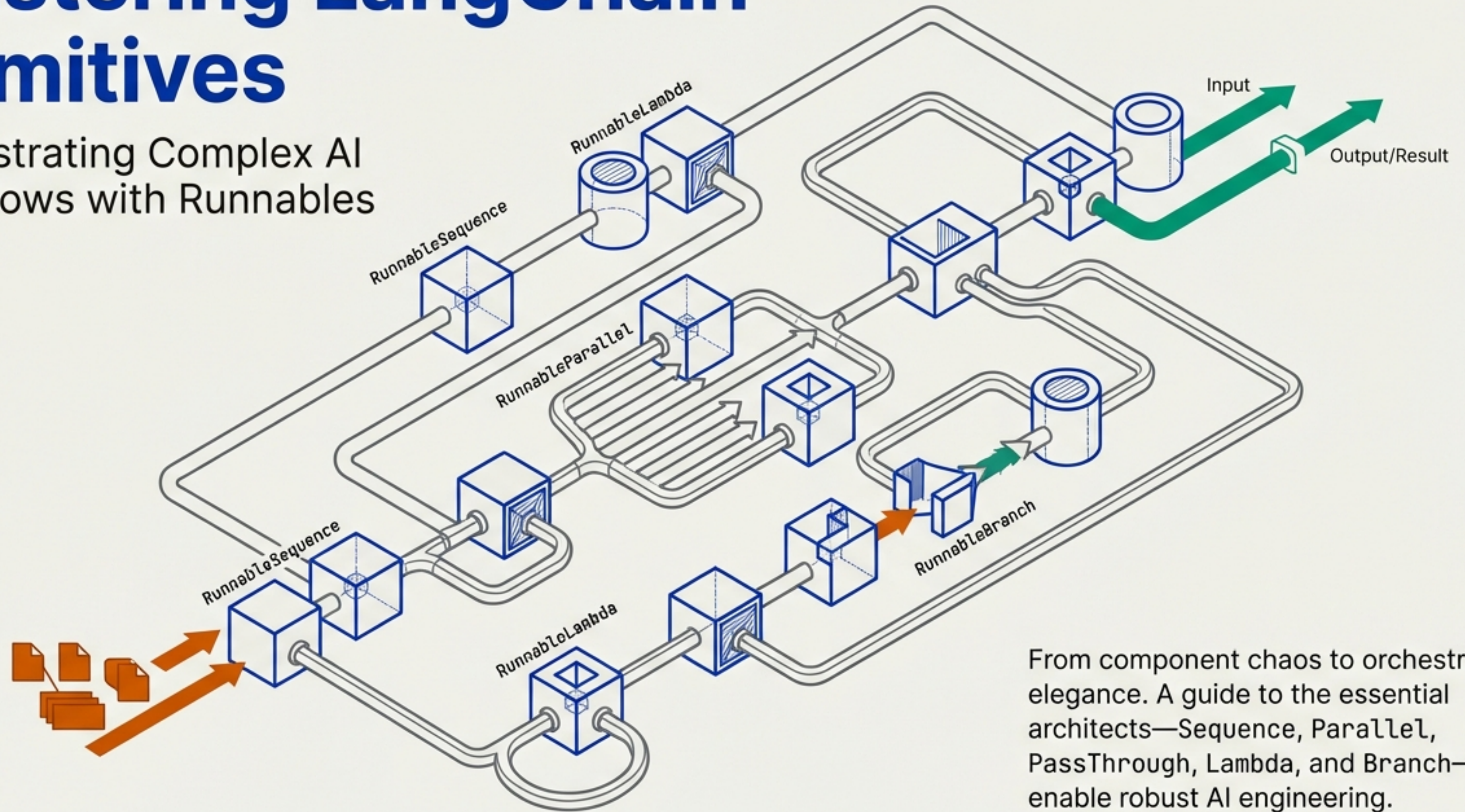


Mastering LangChain Primitives

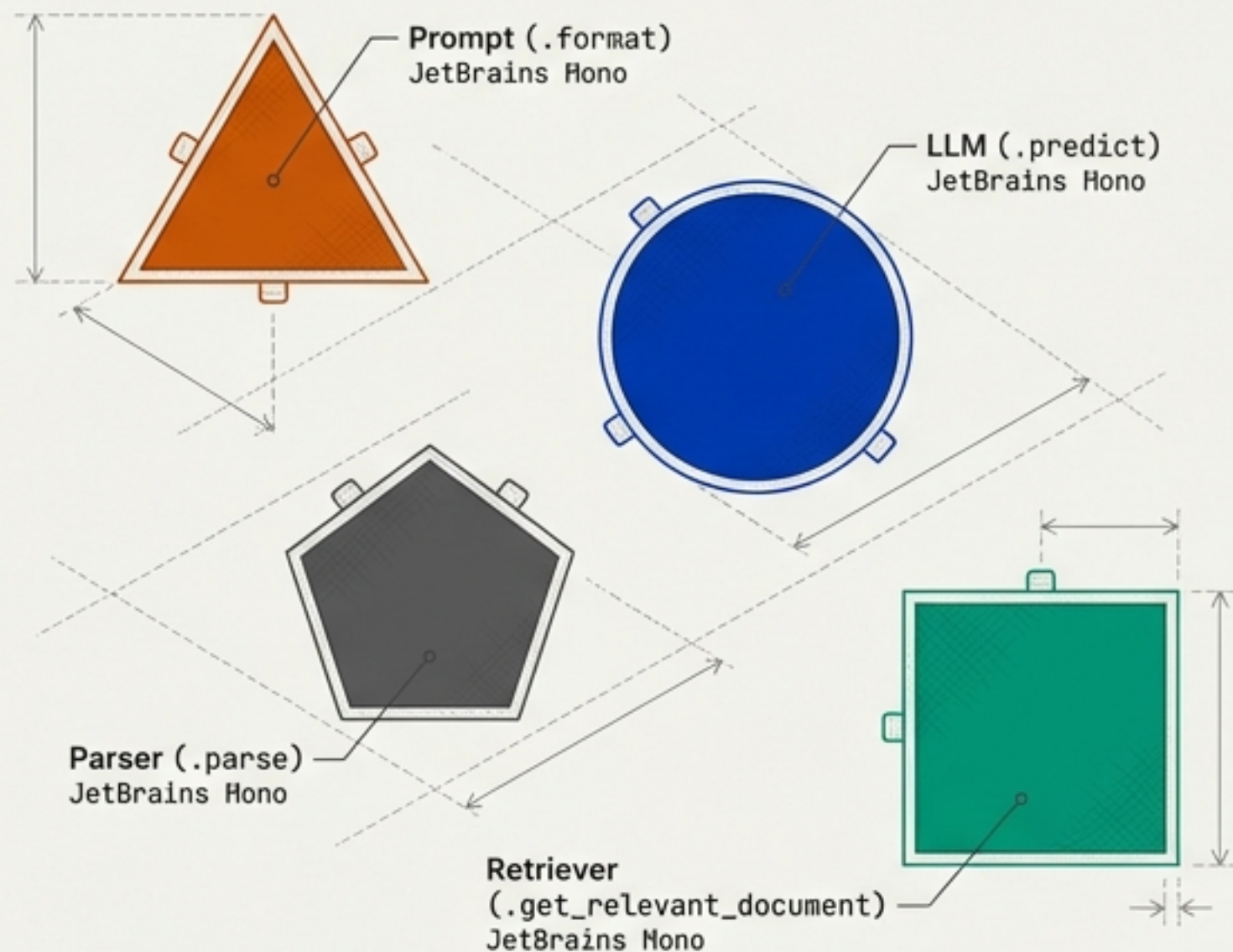
Orchestrating Complex AI Workflows with Runnables



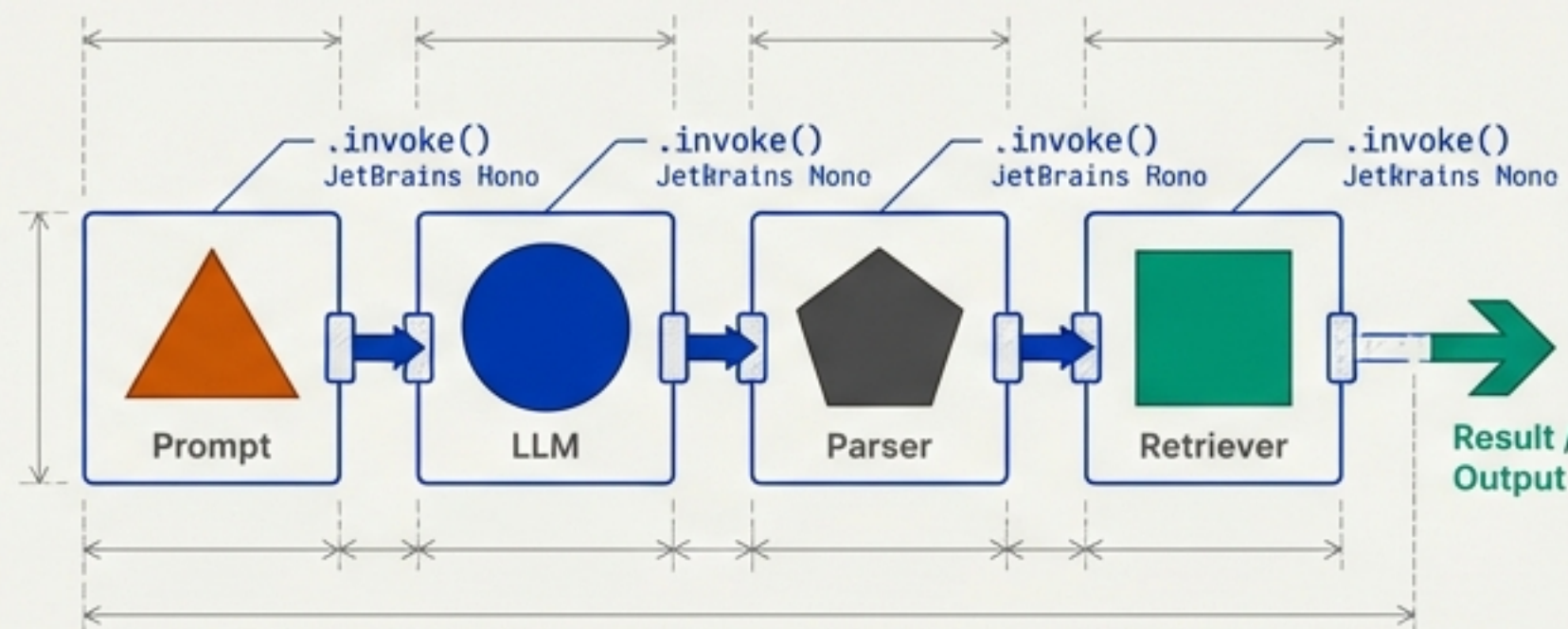
From component chaos to orchestrated elegance. A guide to the essential architects—Sequence, Parallel, PassThrough, Lambda, and Branch—that enable robust AI engineering.

The Evolution of LangChain Architecture

The Fragmentation (Before)

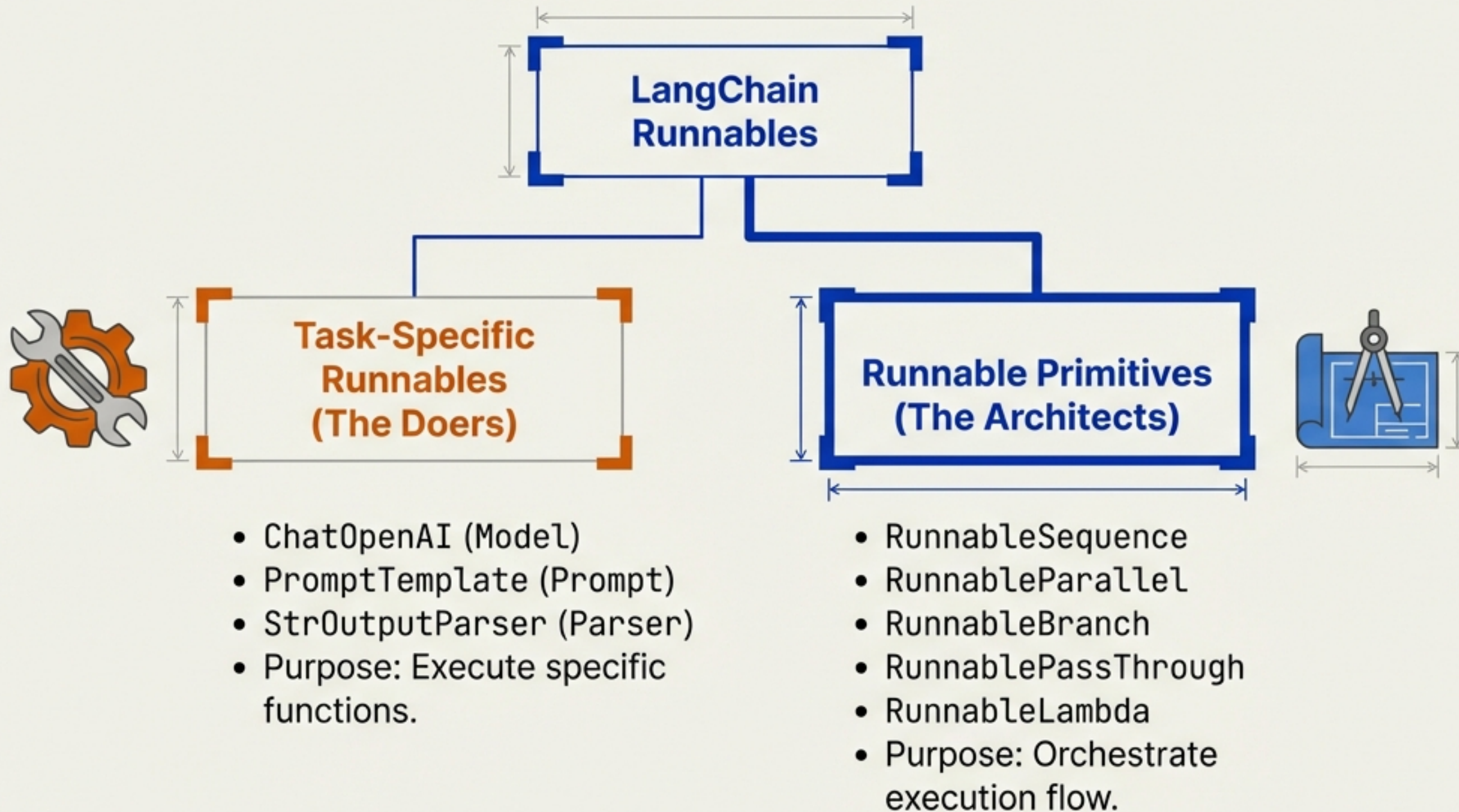


The Standardisation (After)



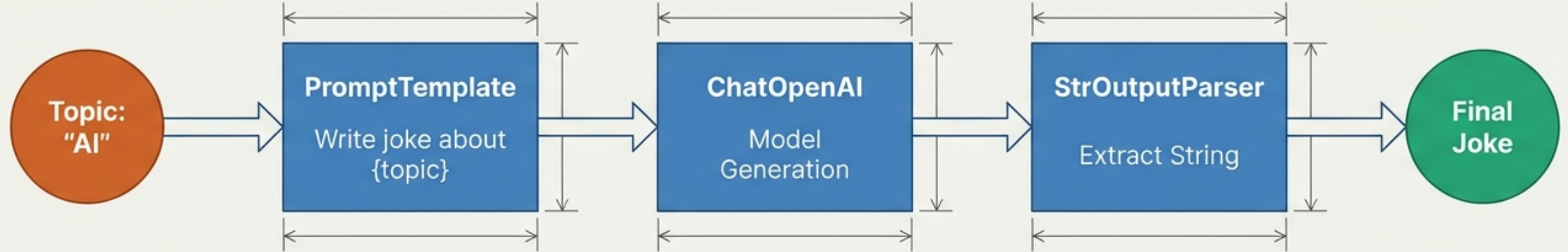
Early LangChain components lacked a unified interface, making custom workflows painful to build. The solution was the Runnable protocol. By enforcing a standard `.invoke()` method, every component became a compatible building block.

The Ecosystem: Doers vs. Architects



We must distinguish between components that perform work and primitives that structure it. Today's focus is on the Architects.

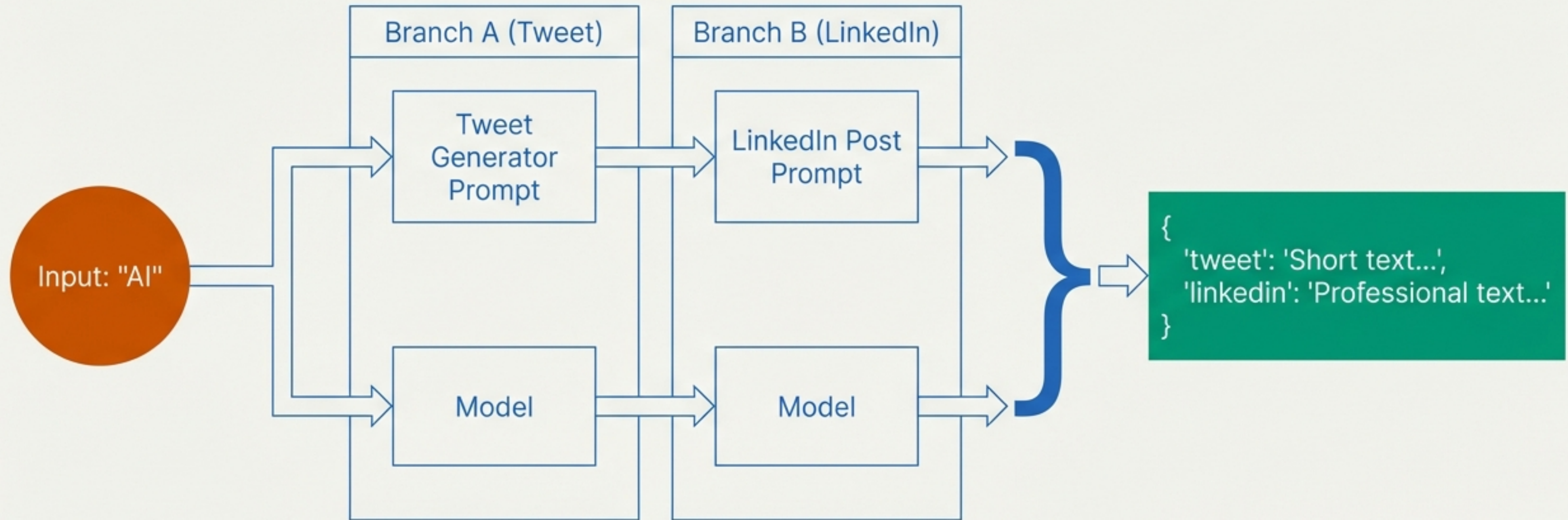
The Linear Flow: RunnableSequence



```
chain = RunnableSequence(  
    first=prompt,  
    middle=[model],  
    last=parser  
)  
chain.invoke({'topic': 'AI'})
```

The most fundamental primitive. Output of Step 1 becomes Input of Step 2. Essential for standard chains.

Multitasking: RunnableParallel

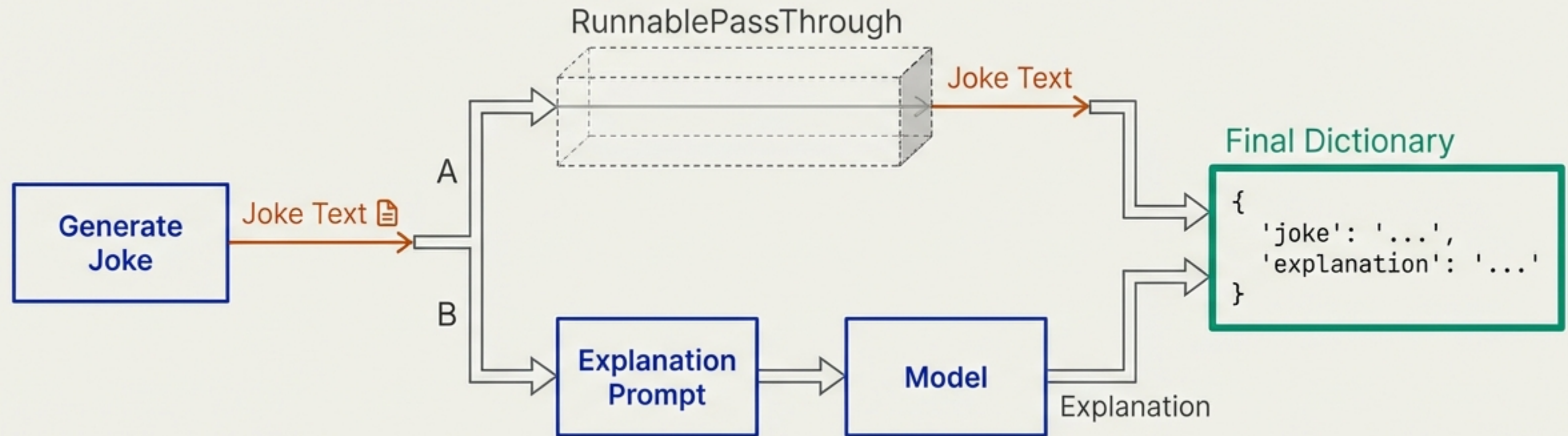


Executes multiple Runnables independently on the same input.
Useful for generating different assets simultaneously.

```
map_chain = RunnableParallel(  
    tweet=tweet_chain,  
    linkedin=linkedin_chain  
)
```

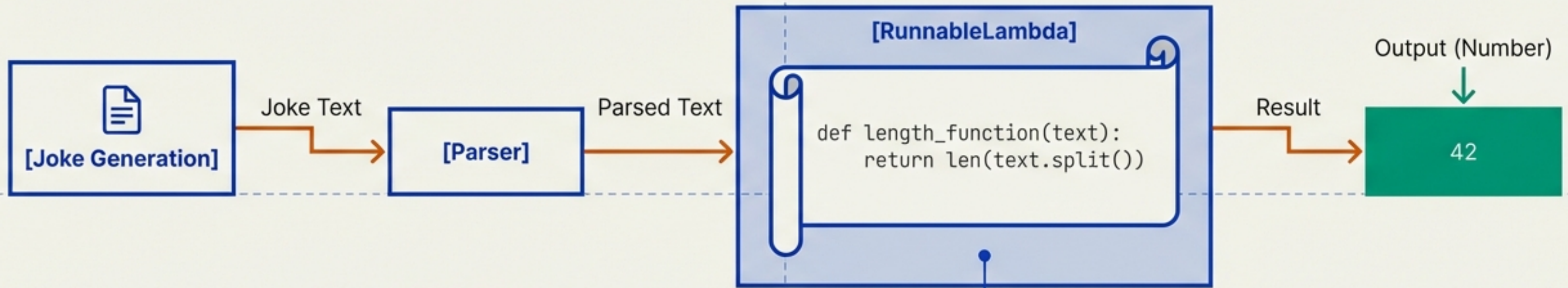

Preserving Context: RunnablePassThrough

Problem: Linear chains overwrite data at every step. How do we keep the original input for later use?



PassThrough acts as a bridge. It allows data to bypass a processing step (like generation) to be rejoined with the output later.

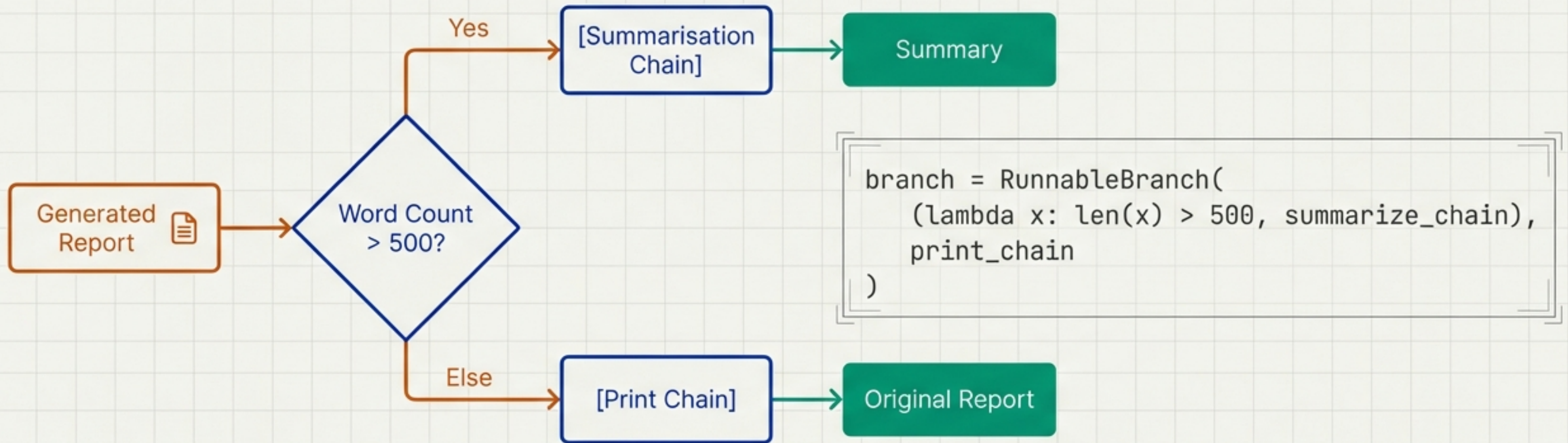
Custom Logic: RunnableLambda



When LLMs are overkill. RunnableLambda wraps standard Python (math, string manipulation, API calls) into a Runnable interface, allowing custom code to exist within the chain.

```
def length_function(text):  
    return len(text.split())
```


Conditional Routing: RunnableBranch



The 'If/Else' of LangChain. Routes execution dynamically based on criteria defined in (Condition, Runnable) tuples.

The Modern Syntax: LCEL

LangChain Expression Language

(The Old Way)

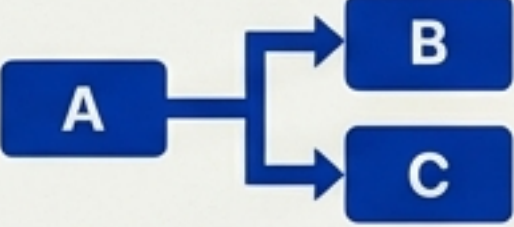
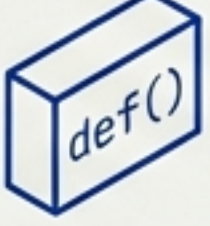
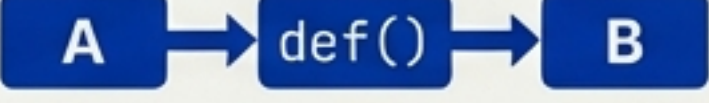
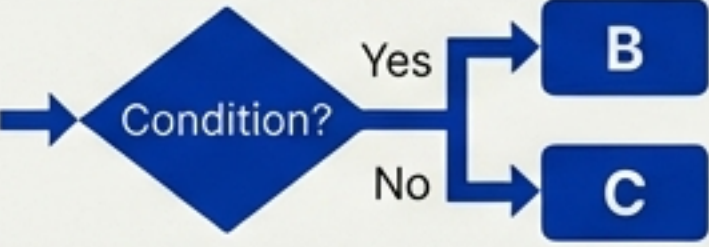
```
RunnableSequence(  
    prompt,  
    model,  
    parser  
)
```

(The LCEL Way)

prompt | model | parser

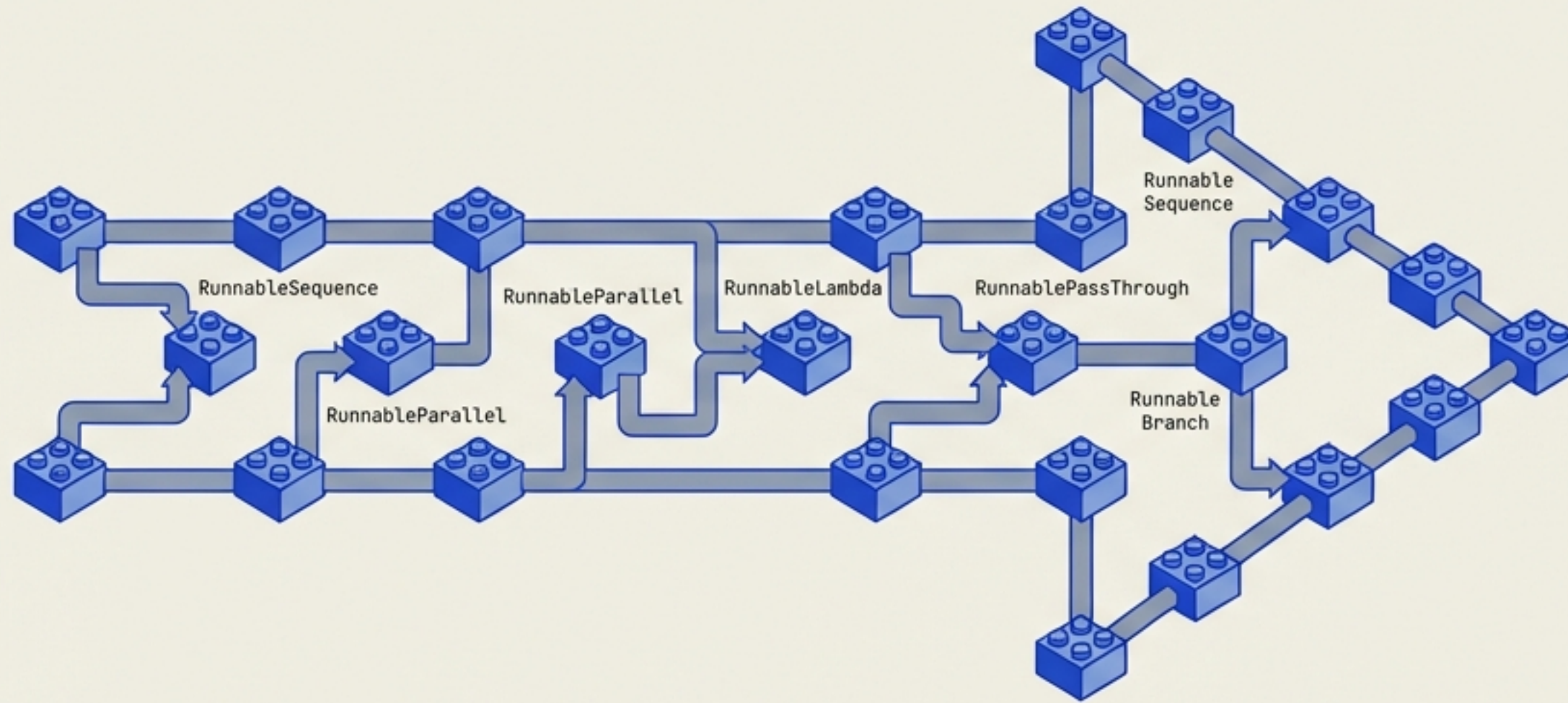
Declarative, readable, standard. The Pipe Operator (`|`) replaces verbose class definitions, becoming the industry standard for defining chains.

The Primitives Cheatsheet

	RunnableSequence Inter Regular	Chain steps linearly JetBrains Mono	
	RunnableParallel Inter Regular	Run simultaneously JetBrains Mono	
	RunnablePassThrough Inter Regular	Preserve Context JetBrains Mono	
	RunnableLambda Inter Regular	Custom Python Code JetBrains Mono	
	RunnableBranch Inter Regular	Conditional Logic JetBrains Mono	

These primitives are the glue that turns isolated components into intelligent applications.

Orchestrating the Future



RAG (Retrieval Augmented Generation)

Retrieval Augmented Generation

We have moved from simple execution to complex orchestration. By mastering primitives, you can build workflows that think, decide, and multitask.

“The power of LangChain lies not just in the models, but in how you connect them.”
Next Step: Experiment with combining Primitives to solve complex logic problems.